

Shopping Data Analysis Based on Amazon

Summary

In this information age, online shopping has been popular with the general public, and user reviews have become a two-way feedback medium for buyers and sellers. The development of emerging technologies expands the frontier of data analysis. Therefore, we can build models to analyze user comments, creating positive benefits for society.

Firstly, we analyzed the text of Appendix and Appendix based on Python, counted the word frequency and drew the word cloud map. We first directly imported reviewText and summary into the program and got the expected results, which proved that the algorithm was correct. Then, in order to retain valuable information, we added the steps of removing numbers, punctuation and stop words to correct. Appendix I users focused on entertainment-related items, and Appendix II users focused on car-related items, both of which were concerned about quality and content.

Secondly, based on the **LDA-K-TF-IDF model**, we performed semantic analysis on Appendix III and Appendix IV, predicted the commodity name and visualized it. After data processing, in order to overcome the limitations of a single model and maximize the reduction error, we successively used **LDA model** and **K-means clustering algorithm** to generate different topics and clusters and visualize them respectively. Then we combined the two models with **TF-IDF** to adjust the parameters to define the name in a narrow and accurate range. And the product name corresponding to each asin is inferred and visualized. The products in Appendix III mainly include office products, and the products in Appendix IV mainly include Musical Instruments and their accessories, cables and other products.

Thirdly, we performed sentiment analysis in Appendices and based on the **improved Naive Bayes model**, inferring the closest score to 'overall' and visualizing it. We performed scoring based on the **LSTM algorithm** in deep learning and the Naive Bayes model, and concluded that the Naive Bayes model combined with the **bag of words and grid search** had the lowest bias. For further optimization, we used **Monte Carlo simulation** to find the weight with the smallest deviation degree, and concluded that the optimal weight for the summary part was 0.5.

Fourthly, we establish 4 evaluation criteria to distinguish machine reviews and customer reviews, and design an algorithm model to calculate the score by combining **syntax tree and entropy weight method**. It is concluded that the score of customer reviews is generally higher than that of machine reviews, which strongly verifies our criteria.

Finally, we conducted a sensitivity analysis of the improved Naive Bayes model and concluded that the confusion of **7.52%** of the data in the training set only caused a **3.21%** increase in the deviation of the prediction results, so our model has excellent stability.

Keywords: shopping review ; the LDA-K-TF-IDF model ; Naive Bayes ; Monte Carlo

Contents

1	Introduction	4
1.1	Problem Background	4
1.2	Restatement of Questions	4
2	Assumptions and Justifications	5
3	Notations	5
4	Task 1:Text Analysis of Commodity Reviews	6
4.1	Ideas and Solutions	6
4.2	Data Visualization and Analysis(Before Correction)	6
4.3	Data Visualization and Analysis(After Correction)	9
5	Task 2: Semantic Analysis of Commodity Reviews	10
5.1	General Idea Analysis	10
5.2	Review Data Preprocessing	11
5.3	LDA (Latent Dirichlet Allocation)	12
5.3.1	Analysis of Model Principle	12
5.3.2	Programming Implementation and Visualization of Results	13
5.4	K-means Clustering Algorithm	14
5.4.1	Analysis of Model Principle	14
5.4.2	Programming Implementation and Visualization of Results	15
5.5	Model-based Product Name Inference Results	16
6	Task 3:Sentiment Analysis of Product Reviews	17
6.1	Problem Analysis	17
6.2	LSTM Algorithm	17
6.2.1	Analysis of Model Principle	17
6.2.2	Programming Implementation and Result Analysis	18
6.3	Naive Bayes	18

6.3.1	Analysis of Model Principle	18
6.3.2	Programming Implementation and Result Analysis	19
6.4	Comparative Analysis and Training Set Correction	20
6.4.1	Comparison and Analysis of Two Models	20
6.4.2	Training Set Correction	21
7	Task 4:Comment Recognition	22
7.1	Model Establishment and Solution	22
7.2	Result Interpretation	24
8	Sensitivity Analysis	24
9	Strengths and Weaknesses	25
9.1	Strengths	25
9.2	Weakness	25
10	Conclusion	25
	References	26
	Appendices	26

1 Introduction

1.1 Problem Background

With the advent of the information age, online shopping is on the rise, and customers often shop on e-commerce platforms. A positive review can bring in new customers, while a negative review can make many people skeptical and thus reluctant to make a purchase. From the consumer's point of view, e-commerce platforms have detailed product introductions and a complete review system. Browsing buyer reviews can provide a basis for consumer decision-making. From the perspective of brand merchants, e-commerce reviews can provide users' evaluation on pricing, product quality, customer service, logistics and other issues as well as preference information, so as to develop more appropriate marketing strategies.

1.2 Restatement of Questions

In the field of e-commerce, AI can help enterprises analyze and mine the value of textual data in real front-line consumer assessments, and digital upgrading of consumer insights in the form of intuitive visualization has been achieved. For example, some researchers fine-tune the BERT model to classify reviews from an Amazon review dataset based on certain characteristics.[1]Deep learning based natural language processing (NLP) can perform keyword extraction, typical opinion mining, and sentiment orientation analysis (positive, negative, neutral) on the indicators mentioned in the feedback data (review text) of customer experience. The available data include review numbers, rating results, and review content for several products. We will use the relevant data in the attachment obtained by the crawler for data statistics and analysis. In order to consider the problem as comprehensively as possible, we will divide the problems to be solved into the following four:

Problem 1: Establish a mathematical model for text analysis of commodity reviews, count the frequency of words in reviews, use the comments in Appendix I and Appendix II to draw word cloud maps, and conduct data and information visualization and analysis.

Problem 2: Establish a mathematical model for semantic analysis of commodity reviews, extract keywords from reviews, predict what commodity names correspond to Appendix III and Appendix IV, respectively, and conduct data and information visualization and analysis of the prediction results.

Problem 3: Establish a mathematical model for sentiment analysis of product reviews, extract keywords from reviews, predict product ratings in Appendix V and Appendix VI, and compare and analyze the results with actual ratings.

Problem 4: Establish several evaluation criteria to distinguish whether a product review is a customer review or a machine review, and verify the evaluation criteria.

2 Assumptions and Justifications

In order to simplify the problem, we present the following model assumptions and their justification:

- **Assumption 1:** The data in the attachment are objective and true. They are comments made by people and have not been tampered with or forged in any way.

Justification: Since the data comes from the Amazon shopping platform, and the reviews on the shopping platform are evaluated by users based on their own experience and feelings, they are credible, real and effective, and can be analyzed to reflect the objective situation.

- **Assumption 2:** A user's comments will not be disturbed by the emotions of other users' comments. Everyone is a highly rational individual and behaves rationally.

Justification: Since people are often driven by others in the evaluation of e-commerce platforms, which leads to wrong judgment, our model will be based on the ideal situation, that is, there will be no interference between each person's behavior, which can greatly reduce the error and reflect the real situation of the product.

3 Notations

Symbols	Description
n	the n -th review data
t	the t -th product
$p(\theta)$	<i>Dirichlet</i> distribution
$p(\theta, z, w \alpha, \beta)$	the joint probability of <i>LDA</i>
J	the loss function for $K - means$
μ_k	the k -th central position($k = 1, 2, 3, \dots, t$)
c_i	the i -th sample($i = 1, 2, 3, \dots, n$)
c_t	LSTM cell state of t moment($t = 1, 2, 3, \dots, n$)
h_t	LSTM hidden state of t moment($t = 1, 2, 3, \dots, n$)
$f(x)$	the <i>NaiveBayes</i> classifier
$\bar{E}$	the degree of deviation
S_1	score by sentiment analysis and text length
S_2	grammar richness score
S_p	the overall score of the differentiated review

4 Task 1:Text Analysis of Commodity Reviews

4.1 Ideas and Solutions

We observe that the data in the attachment is stored in the format of a dictionary in Python, so we first divide the data in the attachment and convert it into a regular table form. At the same time, we remove the content of user name, ID and comment date, and only keep the content in each user's reviewText and summary. Then we used Python to convert the Excel file into a list data type, which removed the keys in the dictionary format and reduced the interference of multiple repeated keys. Then we used Python to calculate the frequency of each word and drew the word cloud map. Figure 1 shows the idea of our algorithm for generating word cloud maps.

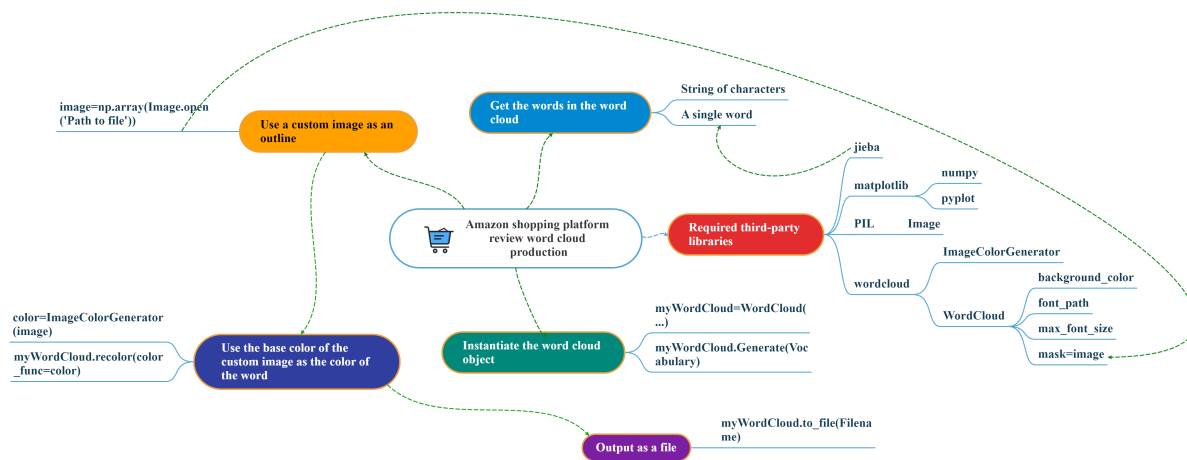


Figure 1: Mind map of word cloud map generation algorithm

4.2 Data Visualization and Analysis(Before Correction)

In order to show the overall situation of the data as comprehensively as possible, we did not initially remove some pronouns, articles, and prepositions, after calculating the frequency of each word in Appendix I and Appendix II, we first analyzed the data in each annex separately, and took the top 30 frequency words to draw a pie chart. The overall situation is presented in the form of word composition and percentage respectively, as shown in Figure 2 and Figure 3. Subsequently, the data in Appendix I and Appendix II are combined to draw the composite pie chart of the top 30 combined frequencies, and the top 10 words are listed separately, which reflects the objective situation of word frequency in the comments on the platform in a more macro way, as shown in Figure 4. At the same time, we take the top 100 words of Appendix I and II frequency and draw them in the form of weight heat map, and form an obvious correspondence, as shown in Figure 5, Figure 6 and Figure 7.

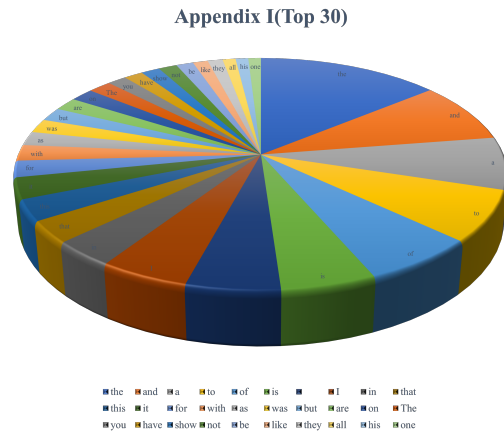


Figure 2: Appendix 1 (Top30)

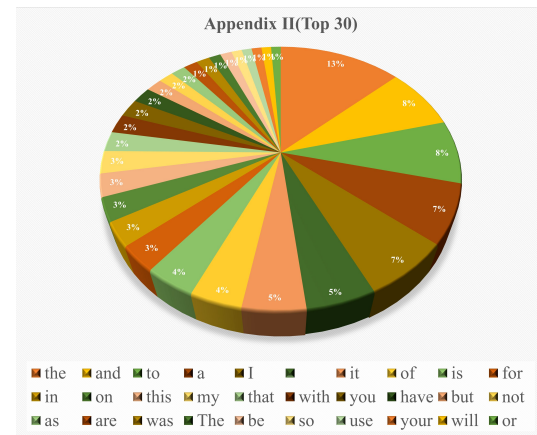


Figure 3: Appendix 2 (Top30)

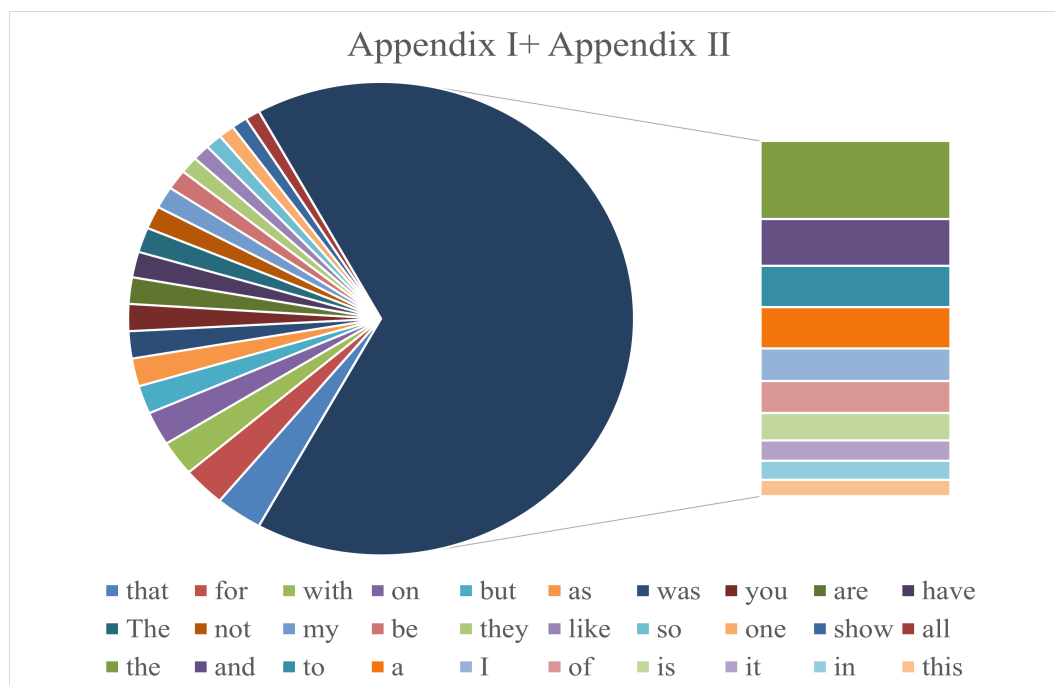


Figure 4: Appendix 1+Appendix 2 (Top30)

As shown in Figure 2, 3 and 4, when we do not deal with stop words such as prepositions, pronouns and articles, the words with the highest frequency in user comments are basically the, and, to, a, I, of and other words that are inevitable in our daily life context, which is consistent with our life experience and vocabulary language characteristics. Therefore, we believe that our word frequency statistical model is reasonable and can be further used to count and rank informative and valuable words such as nouns and adjectives excluding stop words, so as to calculate the frequency of valuable words and draw a real word cloud map highlighting consumption views and emotional tendencies.

the	that	The	just	very	get	had	only	her	even
and	for	not	at	good	by	he	them	which	little
to	with	my	or	out	really	these	there	how	also
a	on	be	This	about	when	do	me	don't	we
I	but	they	from	would	great	their	-	were	it.
of	as	like	has	up	what	see	other	been	any
is	was	so	more	can	who	much	no	first	make
it	you	one	an	some	than	it's	use	time	story
in	are	show	will	his	your	well	into	watch	characters
this	have	all	It	if	season	series	love	because	think

Figure 5: Word Top 100

0.046966	0.00971	0.005494	0.00303	0.00259	0.002173	0.001866	0.001561	0.001441	0.001261
0.028019	0.008837	0.005049	0.002988	0.002575	0.002162	0.001746	0.001542	0.001399	0.001234
0.024986	0.007742	0.004835	0.002986	0.002388	0.002114	0.001734	0.001531	0.001383	0.001221
0.024752	0.007477	0.004423	0.002936	0.002388	0.002086	0.00173	0.001521	0.001378	0.001217
0.019524	0.006225	0.003741	0.002903	0.002321	0.002029	0.001671	0.001517	0.001377	0.001211
0.019243	0.006179	0.003592	0.002828	0.002306	0.00201	0.001651	0.001516	0.001357	0.001197
0.01658	0.006012	0.00346	0.002823	0.002289	0.001934	0.001633	0.00148	0.001337	0.001176
0.012222	0.005965	0.003272	0.002811	0.00227	0.001919	0.001581	0.001472	0.001327	0.001173
0.011415	0.005848	0.00326	0.002764	0.002218	0.001882	0.001567	0.001471	0.001307	0.001166
0.009872	0.005662	0.00304	0.002668	0.002215	0.001869	0.001566	0.001455	0.001289	0.001105

Figure 6: The Frequency of Top 100

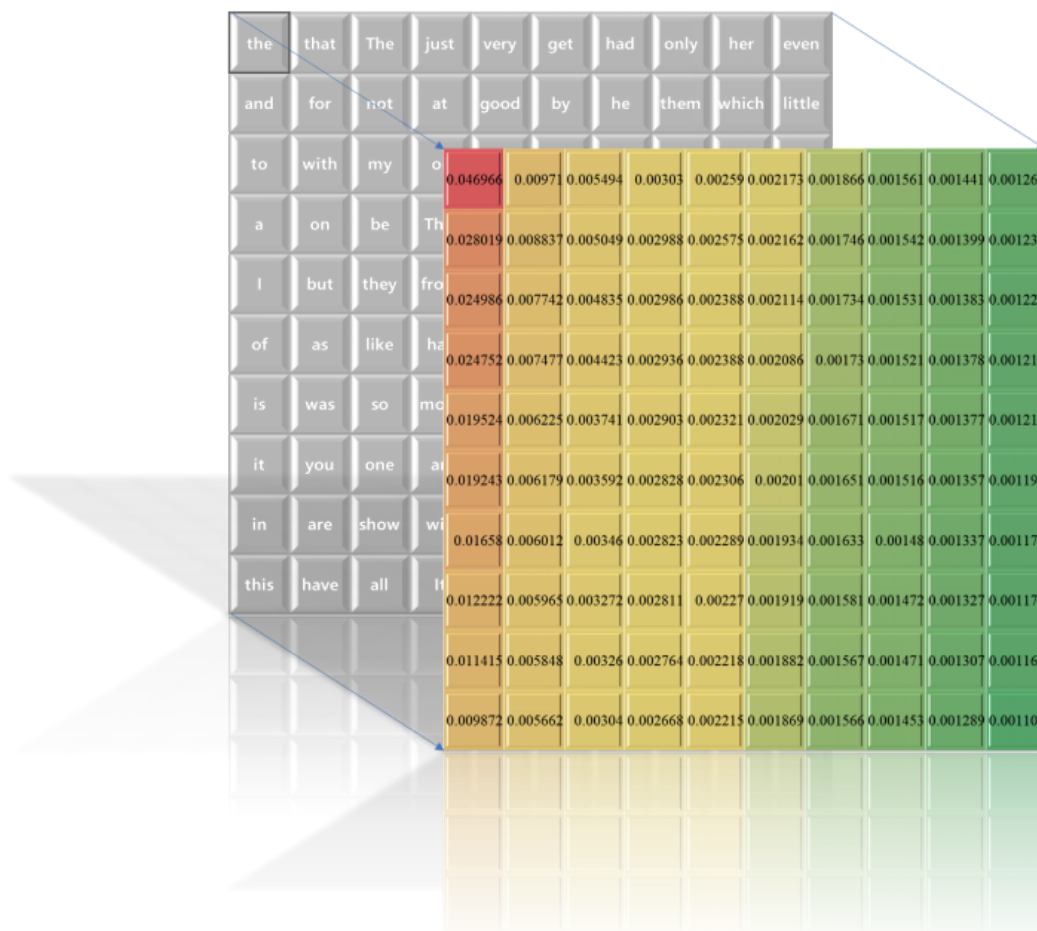


Figure 7: One-to-One Correspondence between Words and Frequencies

4.3 Data Visualization and Analysis(After Correction)

On the previous page, we showed the frequency analysis of Appendix I and Appendix II without removing stop words, as shown in Figures 5, 6 and 7. The results show that the frequency is particularly small in the intuitive case, which leads to a large error. So in the case of removing stop words and missing values as well as numbers and punctuation marks (since question 1 focuses on text analysis and question 2 focuses on semantic analysis, here we present the details of the data processing in question 2), we use the number of frequencies instead of the proportion of frequencies, which greatly reduces the complexity. In this case we derive the frequency ranking as shown in Figures 8 and 9. The word cloud plots is shown in Figures 10, 11, 12 and 13.



Figure 8: Appendix 1(The Corrected Top 20)



Figure 9: Appendix 2(The Corrected Top 20)

According to Figure 8, from these high-frequency words, we can see that reviewers generally pay attention to TV series, movies and other forms of entertainment content (such as show, season, episode, series, movie, film, etc.). In addition, reviewers focus on the quality of the content (e.g., good, great, interesting, bad, etc.), as well as the storyline and characters (e.g., character, story, people, etc.). According to Figure 9, from these high-frequency words, we can see that reviewers generally pay attention to car-related products and services (such as car, battery, oil, tire, filter, etc.). In addition, reviewers also focus on the quality of the product (e.g., good, great, nice, etc.), the experience of using the product (e.g., easy, work, etc.), and the price (e.g., price). Reviewers also mentioned words related to car maintenance (clean, water, etc.).

The LDA-K-TF-IDF model first uses the text mining ability of LDA to model the vocabulary in the document, and divides the vocabulary into multiple topics. Then, K-means clustering is used to cluster the words into multiple clusters, and TF-IDF is used to extract the n most frequent words in each topic and cluster them. Then, by adjusting the number of topics and clusters, the overlap degree of n words in the two models is the highest (the proportion of common words in n words). By keeping each parameter unchanged and taking the intersection, we can obtain the product name of each asin.

5.2 Review Data Preprocessing

Before the semantic analysis of the product review text, we first need to preprocess the data, because we want to establish a mathematical model of semantic analysis, in order to extract the keywords in the review, so as to carry out the theme analysis, to achieve the purpose of inferring the product name according to the content of the product review.

As we know from our daily life experience, the ID and name of each person and the number of the product have no correlation with the content of the product, and the time of each person's comment on the product is not helpful for us to reason about the product name. The content and summary of each person's comment are based on the cognition and feeling of using the product, which makes us make the highest priority indicator. So we first need to remove the content such as reviewerID, asin, helpful, overall, unixReviewTime, reviewTime, etc. and only use the content of reviewText and summary for modeling and text mining.

According to the analysis results of question 1, the most common words in the comment text are the, and, to, I, a, such as articles, prepositions, pronouns, punctuation marks and numbers, etc. These words are not helpful for our modeling, so we should also remove them, so as to further remove the noise in the text after processing. We use lemmatization from spacy to return the word to its original form, reduce the repetition of the word tokenize the comment, then lemmatize it, not only restoring the word form, but also leaving only nouns and adjectives. After that, we transform the word into its normal form. In this way, we have completed the preprocessing of the data and can conduct topic modeling.

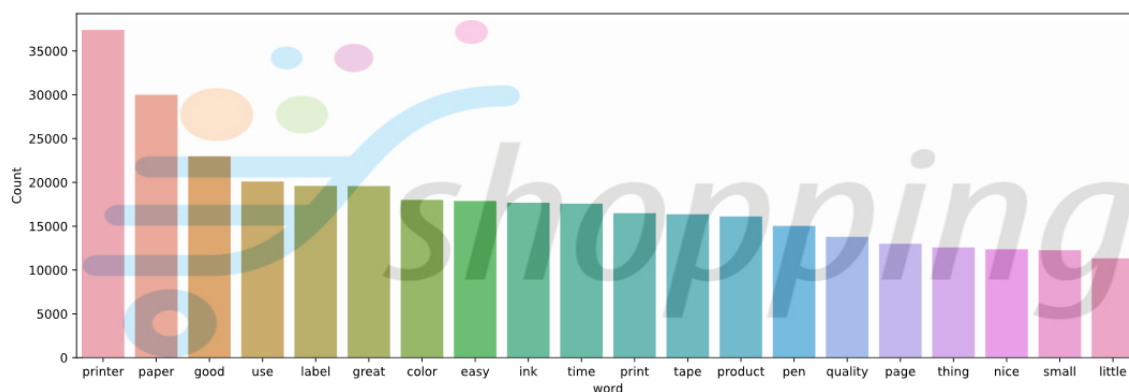


Figure 14: Data Processing Results (Take Appendix 3 as an Example)

5.3 LDA (Latent Dirichlet Allocation)

5.3.1 Analysis of Model Principle

LDA (Latent Dirichlet Distribution) is a generative topic model, and we assume Appendix III is a mixture of multiple topics, each of which in turn consists of multiple words. The goal of the LDA model is to infer the distribution of topics in that file and the distribution of words in each topic based on the Appendix III collection.

Choose parameter $\theta \sim p(\theta)$;

For each N of the words w_n :

- Choose a topic $z_n \sim p(z|\theta)$;
- Choose a word $w_n \sim p(w|z)$;

Where θ is a topic vector, and each column of the vector represents the probability of each topic appearing in Appendix III. The vector is a nonnegative normalized vector. $p(\theta)$ is the distribution of θ , specifically Dirichlet distribution, that is, the distribution of distributions; z_n represents the selected topic, $p(z|\theta)$ represents the probability distribution of topic z given θ , specifically the value of θ , which is $p(z = i|\theta) = \theta_i$.

This method first selects a topic vector θ and determines the probability of each topic being selected. Then, when generating the inner word, a topic z is selected from the topic distribution vector θ , and a word is generated according to the word probability distribution of topic z . [2][3] The graph model is shown as follows:

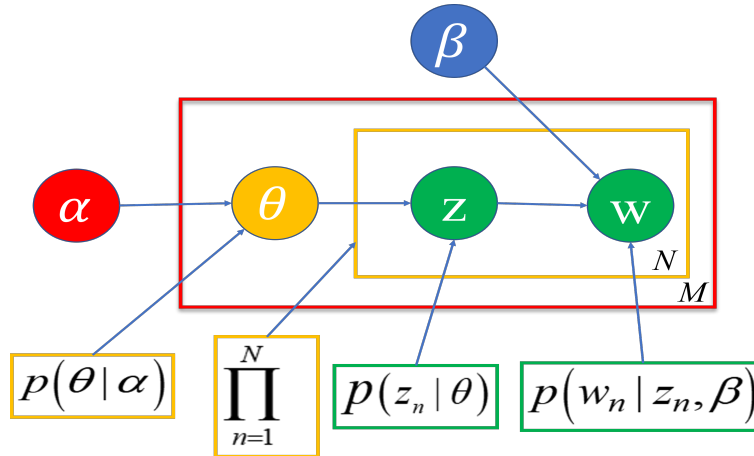


Figure 15: A Graph Model of LDA

From the figure above, we can see the joint probability of LDA:

$$p(\theta, z, w | \alpha, \beta) = p(\theta | \alpha) \prod_{n=1}^N p(z_n | \theta) p(w_n | z_n, \beta) \quad (1)$$

For ease of understanding, the three presentation layers of LDA are shown in three colors:

- Corpus-level (red) : α and β indicate the corpus-level parameters, i.e. each document is the same, so the generation process will only sample once.
- Document-level (orange) : θ is a variable at the document level, with an θ for each document, which means that each document has a different probability of producing each topic z , so generate a θ for each document.
- Word-level (green) : z and w are both word-level variables, z is generated from θ , w is generated from z and β together, and a word w corresponds to a topic z .

From the above discussion on the LDA generation model, it can be known that the LDA model mainly learns and trains two control parameters α and β from the given input corpus. After learning these two control parameters, the model is determined and can be used to generate documents. Where α and β correspond to each of the following information:

- α The distribution $p(\theta)$ requires a vector parameter, the parameter of the Dirichlet distribution, for generating a topic θ vector;
- β Probability distribution matrix of words for each topic $p(w|z)$.

Considering w as the observed variable and θ and z as the hidden variable, α and β can be learned by EM algorithm. In the process of solving, the posterior probability $p(\theta, z|w)$ cannot be directly solved, so a lower bound of likelihood function is needed to approximate the solution. This can be computed using variational inference based on factorization assumptions, using the EM algorithm. Each time the E-step inputs α and β , compute the likelihood function, the M-step maximizes this likelihood function, compute α and β , and iterate until convergence.

5.3.2 Programming Implementation and Visualization of Results

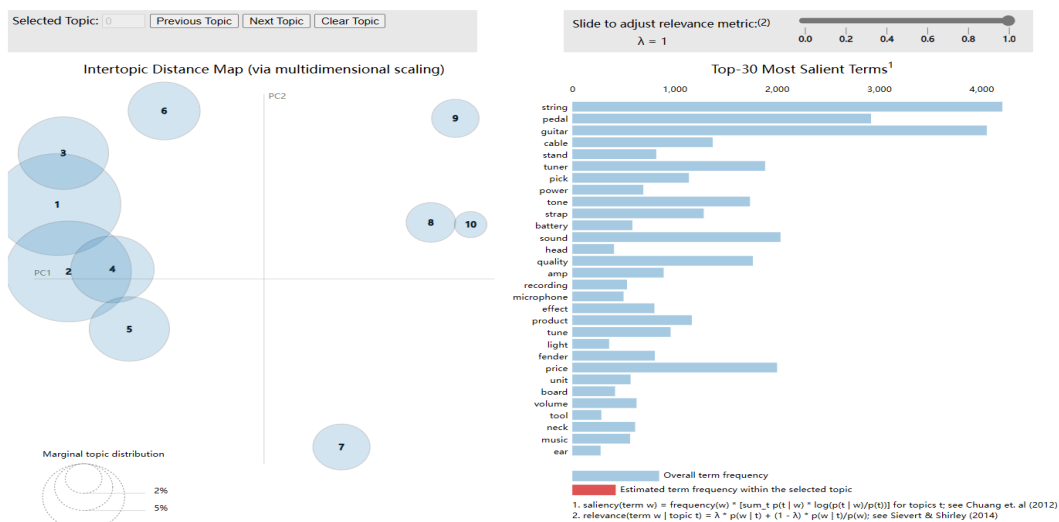


Figure 16: Interactive Visualization of Topic Modeling Data

Before visualizing the data, we ran the program and found that a thorough topic modeling of the data in the appendix would generate 2,420 topics. Due to the time complexity of the code and the space complexity of the vector plots, we found that the amount of data in Appendix III and Appendix IV would generate 900 topics. Because we need to specify the number of topics generated during the execution of the algorithm, that is, the *num_topics* parameter needs to be set manually, we first calculate the *asin* code in Appendix IV. The same *asin* code is the same product, but different *asin* codes also have the possibility of the same product, too few topics will increase the error. However, too many topics can lead to overfitting, so we adjusted the parameter and got a good result with *num_topics*=100 for visualization. The topic distribution is shown in Figure 16. The processing method is the same in Appendix III, and the final results will be shown in Section 5.5. The data visualization on the previous page is interactive, but because it is an html file, it can only be opened and manipulated on a web page, so we only show the overall picture of the subject.

5.4 K-means Clustering Algorithm

5.4.1 Analysis of Model Principle

After the problem analysis in Part 5.1, it can be known that the data in Appendix III and Appendix IV belong to the training set without labels, so the supervised learning method is not feasible to some extent. We intend to use K-means clustering in unsupervised learning to vectorize the text data and then reduce the dimension. In k-means clustering, without knowing any sample labels in advance, the samples are divided into several categories through the internal relationship between the data, so that the similarity between samples of the same class is high and the similarity between samples of different classes is low (that is, the class cohesion is increased and the distance between the classes is reduced). Its core idea is to iteratively find a partitioning scheme for the K clusters that minimizes the loss function corresponding to the clustering result[4]. We first perform the convergence proof, where the loss function can be defined as follows:

$$J = \sum_{i=1}^C \sum_{j=1}^N r_{ij} \cdot v(x_j, \mu_i) \quad (2)$$

Where is:

$$v(x_j, \mu_i) = \|x_j - \mu_i\|^2, r_{nk} = \begin{cases} 1 & \text{if } x_n \in k, \\ 0 & \text{else.} \end{cases} \quad (3)$$

To find the extremum, we set the partial derivative of the loss function equal to 0:

$$\frac{\partial J}{\partial \mu_k} = 2 \sum_{i=1}^N r_{ik} (x_i - \mu_k) = 0 \quad (4)$$

When k is the KTH center, we have:

$$\mu_k = \frac{\sum_{i=1}^N r_{ik} x_i}{\sum_{i=1}^N r_{ik}} \quad (5)$$

It can be seen that the new center point is the centroid of all the classes.

The core goal of k-means is to partition a given dataset into K clusters (where K is a hyperparameter) and give a center point corresponding to each sample. The steps can be divided into four steps:

1. Data preprocessing. Here we vectorize the text data in Appendix IV numerically and then preprocess the data.
2. Randomly select K centers, denoted as: $\mu_1^{(0)}, \mu_2^{(0)}, \dots, \mu_k^{(0)}$
3. Define the loss function: $J(c, \mu) = \min \sum_{i=1}^M \|x_i - \mu_{c_i}\|^2$
4. Let $t = 0, 1, 2, \dots$. Is the number of iteration steps, repeat the following process until J converges:
 - For each sample x_i , assign it to the nearest center $c_i^t < -\arg \min_k \|x_i - \mu_k^t\|^2$
 - For each center k , recalculate the center of the class $\mu_k^{(t+1)} < -\arg \min_{\mu} \sum_{i: c_i^t = k} \|x_i - \mu\|^2$

5.4.2 Programming Implementation and Visualization of Results

In the programming implementation of the model solution, we first use the TF-IDF method to numerically vectorize the data processed text, and then specify the number of clusters. In order to be consistent with the LDA model topic parameters in Section 5.3, we also use the parameter setting of `n_clusters=100` here. In order to show the results visually, we use principal Component Analysis (PCA) to reduce the dimension of TF-IDF matrix to two-dimensional space, and then draw a scatter plot, which effectively avoids the problem of inconsistent array size and retains the actual results of K-means clustering. The final clustering situation is shown in Figure 18. Good results are obtained.

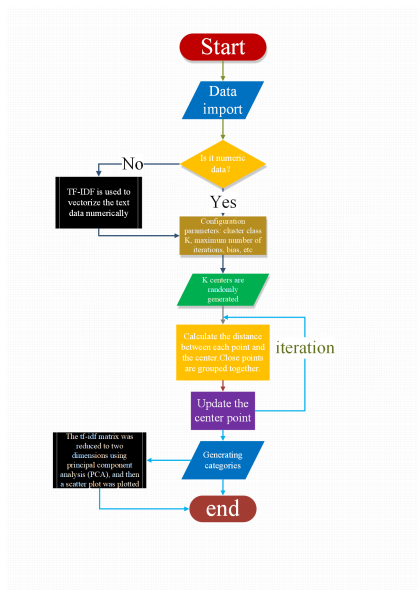


Figure 17: K-Flow Chart

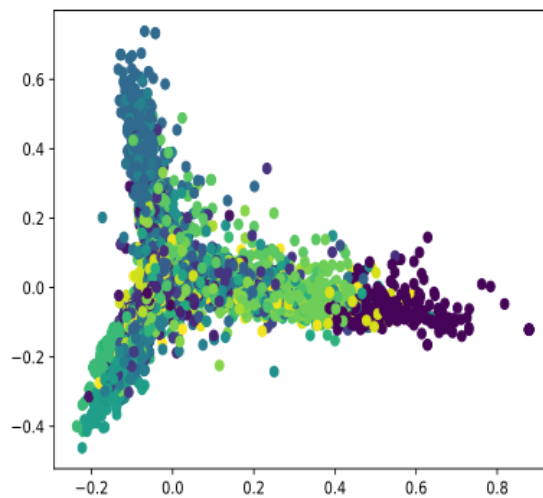


Figure 18: Distribution of Clustering Results

5.5 Model-based Product Name Inference Results

Based on the LDA-K-TF-IDF model, following the ideas mentioned in Section 5.1, we finally defined the product names in a small but accurate range and deduced the most likely product names. Due to space reasons, we only show the asin code and name of some products, the results are shown in Figure 19, and the word cloud map of the overall product name is drawn, as shown in Figure 20 and 21. It can be concluded that the total number of products in Appendix III is 2420, the total number of products in Appendix IV is 900, and most of the products in Appendix III are paper, printer, tape, pencil, calculator and other products, mainly office products, Appendix IV mainly guitar, cable, pedal, tuner, etc., mainly Musical Instruments and their accessories and cables, etc.

Order	Asin	Product Name (in Yellow)			Order	Asin	Product Name (in Yellow)		
1	B00000JBLH	calculator	product	year	1	1384719342	pop	filter	product
2	B00000JBLU	model	calculator	battery	2	B00004Y2UT	cable	model	guitar
3	B00000JBX	calculator	math	class	3	B00005ML71	pedal	keyboard	use
4	B00000JBO8	calculator	school	lot	4	B000068NSX	cable	cord	month
5	B00000JFNV	card	paper	printer	5	B000068NTU	cable	quality	connection
6	B00000JZKB	calculator	use	user	6	B000068NVI	cable	cord	price
7	B00000K3T0	calculator	key	display	7	B000068NW5	cable	quality	price
8	B00001MXVQ	wrist	product	day	8	B000068NXC	wire	cable	microphone
9	B00002NDS2	tape	label	product	9	B000068NXC	cable	adapter	stereo
10	B00002QUKW	label	time	size	10	B000068O1N	cable	price	mixer
...	...	...	...	...	...	...	...	...	...
2411	B00JXLGESY	printer	print	paper	891	B00EPWASQ0	pedal	tone	sound
2412	B00JXLGETI	printer	print	paper	892	B00EQDITGO	pedal	battery	loop
2413	B00JXLGETS	printer	print	paper	893	B00F2PKRJW	drum	sound	year
2414	B00JXLGEUC	printer	paper	print	894	B00FXKIG5I	tuner	neck	use
2415	B00JXLGF06	printer	paper	print	895	B00GTSM8FW	string	guitar	capo
2416	B00K5U2P2S	printer	color	print	896	B00H02C9TG	quality	device	input
2417	B00KLLBEP1	paper	line	quality	897	B00HFRXACG	string	bass	tone
2418	B00KOO594O	box	key	document	898	B00IZCSW3M	string	guitar	tone
2419	B00KOO599O	box	product	document	899	B00J4TBMVO	string	guitar	tone
2420	B00KYAORC2	scale	display	ounce	900	B00JBIVXGC	string	guitar	tone

Figure 19: Appendices 3 and 4 Commodity Name Prediction Results (partial)



Figure 20: Overall Product Name(3)



Figure 21: Overall Product Name(4)

6 Task 3:Sentiment Analysis of Product Reviews

6.1 Problem Analysis

Similar to problem 2, the solution of problem 3 still requires quantification and scoring of words. But the third question is to relate each comment to the corresponding rating. In order to extract keywords from the comments of Appendix V and Appendix VI on the basis of question 2 and further predict product ratings, other appendices should first be used as training sets to extract the comments and rating information preprocessed in question 2, and then combine the two with machine learning algorithms. Apply the trained model to Appendix V and Appendix VI. The prediction accuracy of the model was evaluated by the prediction score and the real score, and the model was improved until a relatively satisfactory answer was obtained.

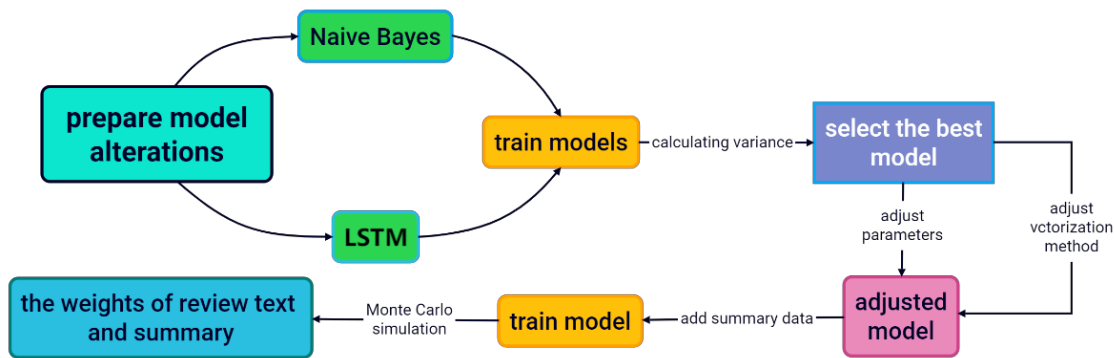
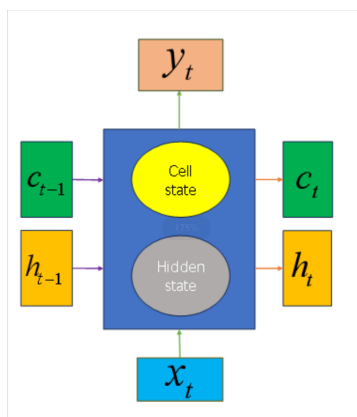


Figure 22: Task 3 Process

6.2 LSTM Algorithm

6.2.1 Analysis of Model Principle



Each LSTM unit consists of a forgetting gate, an input gate, an output gate, and a cell, which contains a cell state and a hidden state[5]. And its internal process is mainly divided into three stages:

1. Forget stage: forgetting door selectively forgets the memory of the previous unit;
2. Memory stage: the input data is selectively stored in each cell;
3. Output stage: Decide to output data and output.

Symbol description:

x_t :the input data y_t :the output data

c_t :cell state of t moment h_t :hidden state of t moment

Figure 23: LSTM Cell Diagram

6.2.2 Programming Implementation and Result Analysis

We used Appendix IV as the training set to train the model. Due to the large amount of total data, and in order to ensure the objective and comprehensive data, we presented the score predictions of the first 30 and the last 30 data in Appendix V.

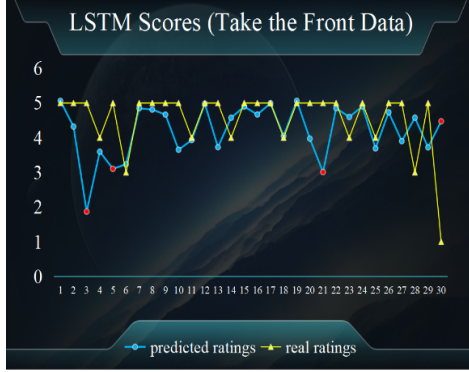


Figure 24: The First 30 Data

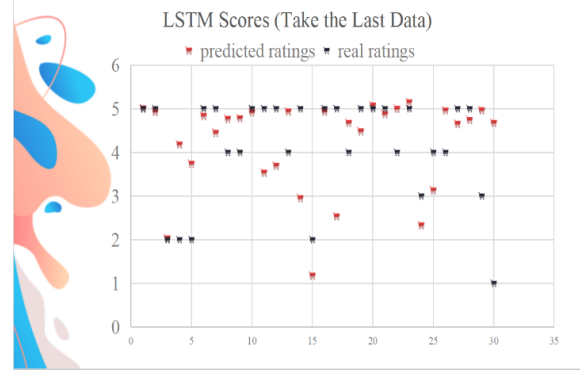


Figure 25: The Last 30 Data

According to the statistical results, among all 60 pieces of data, 18 pieces of predicted ratings have a deviation greater than 1 from the true value, and 8 pieces have a deviation greater than 2, indicating a large degree of deviation.

It can be observed that the prediction accuracy of the model for the comments with high scores is higher than that for the comments with low scores, which is caused by the small number of low scores in the training set.

6.3 Naive Bayes

6.3.1 Analysis of Model Principle

The steps of Naive Bayes' learning algorithm are as follows:

1. Prepare data: that is, transform data into a form that can be used for Naive Bayes classification, such as converting text data into numerical feature vectors. Let each sample x in the dataset contain n -dimensional features, and the class tag set contains k categories, i.e. :

$$\begin{cases} x = (x_1, x_2, \dots, x_n) \\ y = (y_1, y_2, \dots, y_k) \end{cases} \quad (6)$$

2. Training model: Use the training data to estimate the prior probability of each category and the conditional probability of each feature. The Naive Bayes classifier assumes that each feature is independent of each other. According to the above conditions, the Naive Bayes classifier can be expressed as:

$$f(x) = \arg_{y_k} \max P(y_k) \times \prod_{i=1}^n P(x_i|y_k) \quad (7)$$

- Prediction: For each given test sample, the posterior probability of each class is calculated using Bayes' theorem, and the class with the greatest posterior probability is selected as the result.[6]

To assess the accuracy of the forecast we compare the predicted rating with the real rating to calculate the degree of deviation of the prediction. The degree of deviation can be expressed by the following formula:

$$\bar{E} = \frac{\sum \|r_p - r_r\|}{n_{data}} \quad (8)$$

r_p : The predicted score. r_r : The true score. n_{data} : The total number of predicted scores.

6.3.2 Programming Implementation and Result Analysis

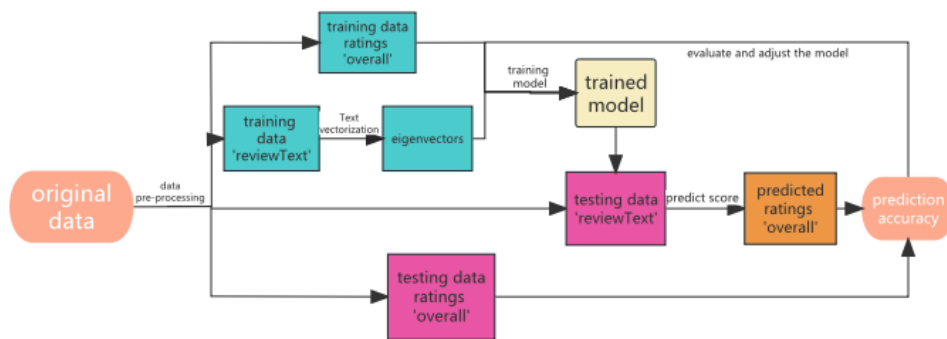


Figure 26: Program Process

When adjusting Naive Bayes, we give two alternatives for text vectorization and model parameter selection, namely TF-IDF model, bag of words model, and whether to use grid search and cross-validation to get adjusted model parameters. By changing the combination relationship of selection and the amount of data used as the training set, the degree of deviation of prediction results of each group can be obtained. The optimal combination is selected according to the degree of deviation.

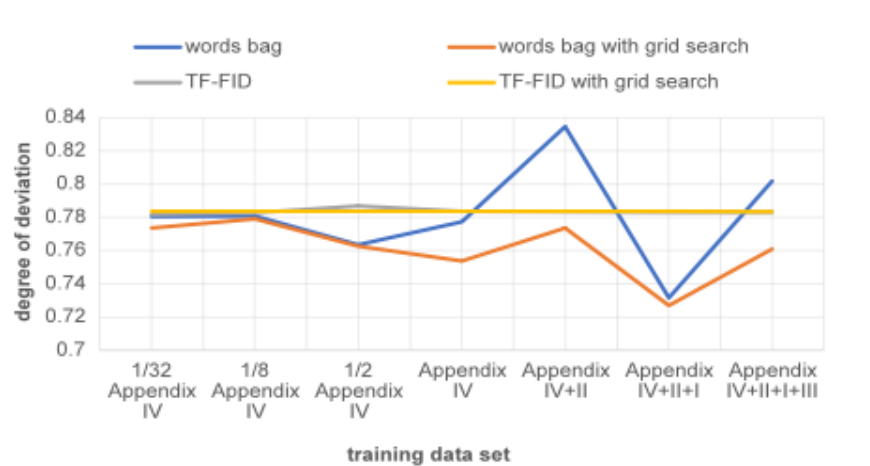


Figure 27: The Deviation Degree of Four Ways to Implement Naive Bayes

- The change in the fluctuation amplitude of group words bag and words bag with grid search is due to the sample size of the first three data is small and all come from Appendix , resulting in similar word bags.
- In general, TF-IDF had a higher accuracy in prediction than words bag, but the average deviation was similar between the two groups, which may be because the training set and prediction set both used the appendices after removing the stop words, and the advantage of TF-IDF in processing the existing words in the general section was not fully utilized.
- The four groups still have relatively low degree of deviation when the training set data is small, indicating that the common advantage of the four groups is that they are not sensitive to the reduction of data.
- Overall, combination words bag with grid search has the lowest degree of deviation, so the Bayesian sub-model of this combination is selected for subsequent operations.

Using the same method as 6.2.2 and applying the model of groupwords bag, Appendix IV is used as the training set, and the first 30 and last 30 data of Appendix V are used as the test set, resulting in the following charts:

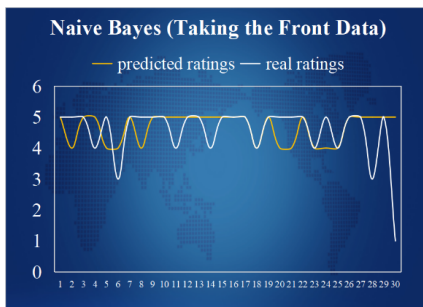


Figure 28: The First 30 Data

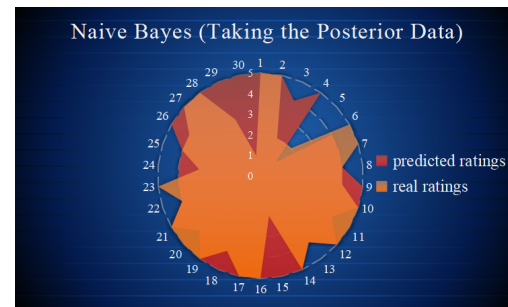


Figure 29: The Last 30 Data

According to the statistical results, among all 60 pieces of data, 8 pieces of predicted ratings have a deviation greater than 1 from the true value, and 3 pieces have a deviation greater than 2, this shows that the prediction results are relatively accurate.

By observing the radar map on the right, it can be found that the graph area of the predicted value is larger than the graph area of the true value, indicating that the above model has a high prediction of the score.

6.4 Comparative Analysis and Training Set Correction

6.4.1 Comparison and Analysis of Two Models

Again, we use Appendix IV as training set. And chose the "words bag with grid search" group with the highest prediction accuracy to represent the Naive Bayes model and make the comparison between and LSTM. We used the first 50 pieces of data in Appendix as the test set to evaluate the degree of prediction deviation of the model.

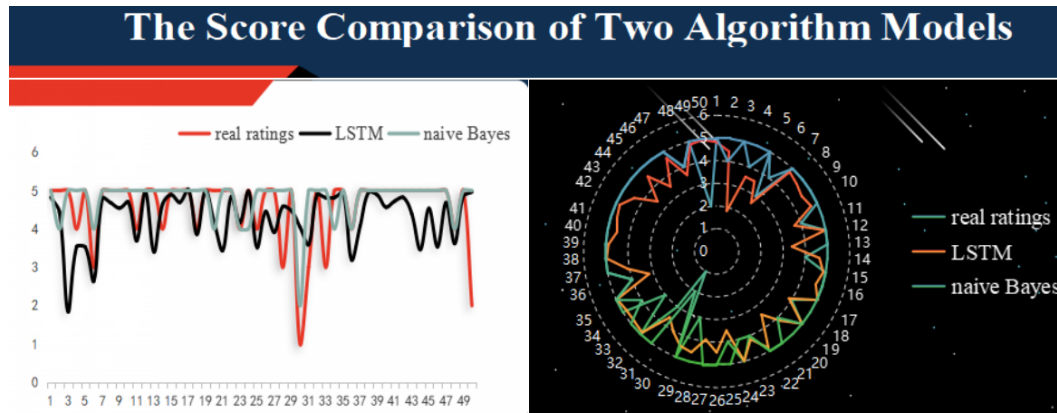


Figure 30: Comparison of Two Models

- It can be seen from the intuitive analysis that the deviation between the prediction results of Naive Bayes model and the true value is smaller than that of LSTM model, and its prediction accuracy is higher than LSTM.
- Limited by its own model, Naive Bayes can only predict integer scores, and the predicted scores are concentrated on high scores. Compared with LSTM, Naive Bayes has the disadvantage of being insensitive to different comments.
- To sum up, we attach importance to prediction accuracy and choose Naive Bayes algorithm for the next step to ignore the sensitivity of the model to different comments.

6.4.2 Training Set Correction

Considering that summary can often truly reflect the emotions of reviewers, we obtained two sets of predicted ratings from review text and summary. The weights were assigned to the two sets of data, and Monte Carlo simulation was used to find the weight that minimized the degree of deviation. The sum of the weights of A and B is one. (In order to reduce deviation, we choose the model words bag with grid search in 6.3.2).

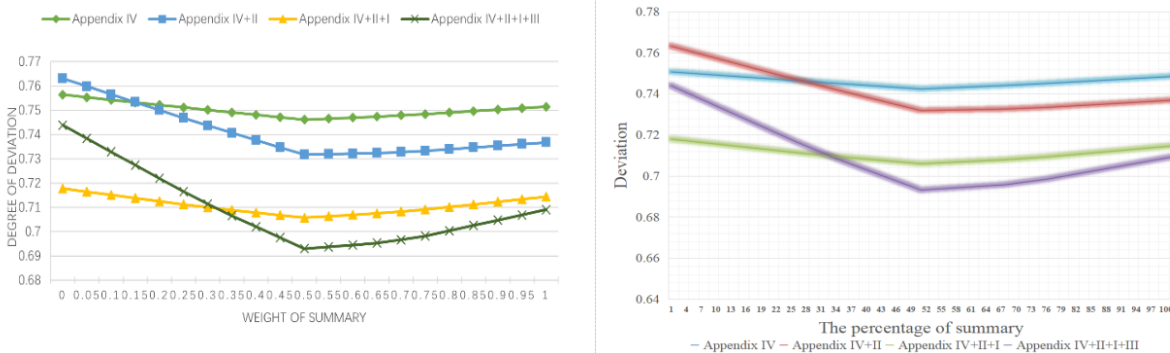


Figure 31: Variation of Deviation Degree with Weight or Percentage

Table 1: Minimum Deviation and Coefficient of Weight

Training set	Appendix 4	Appendix 4+2	Appendix 4+2+1	Appendix 4+2+1+3
Minimum deviation	0.74224	0.7317	0.70579	0.69303
Coefficient of weight	0.50061	0.50009	0.50041	0.50067

When the amount of training set data changes greatly, the optimal weights obtained by Monte Carlo simulation are stable at about 0.5, so it can be seen that the optimal weights are constants independent of the amount of training set data.

Compared with only using review text, the prediction deviation degree of Naive Bayes using both summary and review text under optimal weight is significantly negatively correlated with the training data, and has higher prediction accuracy. We did not input more training sets due to the limited amount of data, but it can be seen that this model still has the potential to improve the prediction accuracy.

Even though summary usually contains a small amount of text, it accounts for 0.5 weight in the model, which is consistent with the reality of life. People's views on products are often clearly reflected in summary. Therefore, it is necessary to add summary to the prediction model.

7 Task 4:Comment Recognition

7.1 Model Establishment and Solution

Our group came up with the following indicators to judge whether a comment is a machine comment:

1. The time of the comment, if a large number of similar comments appear on the same day or several days, then the probability is high.
2. Emotional expression of comments and text length, machine comments often use a large number of sentences to express clear emotions.
3. The richness of sentence syntax, the type of machine sentence syntax is single.
4. The degree of correlation between reviews and products, and the probability of generalized reviews is machine reviews.

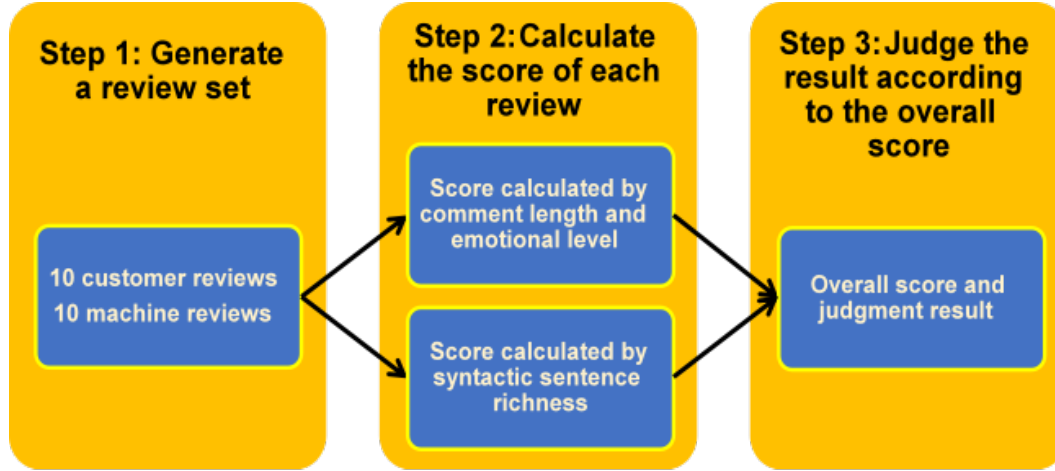


Figure 32: Program Flow Figure

In order to judge the prediction accuracy of the model and determine the parameters of the model, we collected 10 comments from Appendix as human comments, and then generated 10 machine comments by computer.

In consideration of feasibility, we only consider points 2 and 3 above as criteria. Set the overall score as:

$$S_p = S_1 \times w_1 + S_2 \times w_2 \quad (9)$$

S_1 : Score by sentiment analysis and text length.

S_2 : Grammar richness score.

w_1, w_2 : The weight obtained by the entropy weight method.

Where S_1 can be represented by the following formula:

$$S_1 = \frac{n_s}{\sum \|s_e\|} \quad (10)$$

s_e : The emotional value of each sentence obtained by the model of Task .

n_s : The number of sentences in the review.

By using the context-free method and creating a grammar tree, we scored the sentence syntax richness of paragraph comments. The higher the score, the richer the structure, and the score was recorded as S_2 .

7.2 Result Interpretation

	1	2	3	4	5	6	7	8	9	10	11(m)	12(m)	13(m)	14(m)	15(m)	16(m)	17(m)	18(m)	19(m)	20(m)
$S_1(w_1=0.65)$	14.6	5.8	9.8	7.1	6	6.8	12	10.9	12.5	5.3	4.7	1.1	0.9	5.9	3.8	2.3	5	2.4	3.2	4.3
$S_2(w_2=0.35)$	36	33	34	30	46	27	49	35	52	32	15	8	11	23	23	21	18	21	22	11
$S_p(\text{overall})$	22.1	15.3	18.3	15.2	20	13.9	25	19.4	26.4	14.7	8.3	3.5	4.4	11.9	10.5	8.9	9.6	8.9	9.8	6.6

*m represents the review is generated by machine

Figure 33: Score of the Reviews

According to the table, the overall score of the first ten is significantly higher than that of the last ten, which is in line with the expectation, indicating that the model has good judgment ability.

Based on the combined score of each review, we use 12.5 as the criterion, and only reviews with a score higher than 12.5 can be considered customer reviews.

A few of the overall reviews have low scores, which may be due to customers not taking the reviews seriously.

8 Sensitivity Analysis

Since other model properties other than problem three are overly dependent on the text content, it is not convenient for us to make quantitative modifications to the text, so here we only perform sensitivity analysis on the modified Naive Bayes model in model three. Our basic idea is to take the data from Appendix , randomly change the 772 'overall' scores to confuse the model's training set, and then use the trained model to score Appendix and calculate the deviation. Then compare the results with the results before the change to determine the stability of the model.

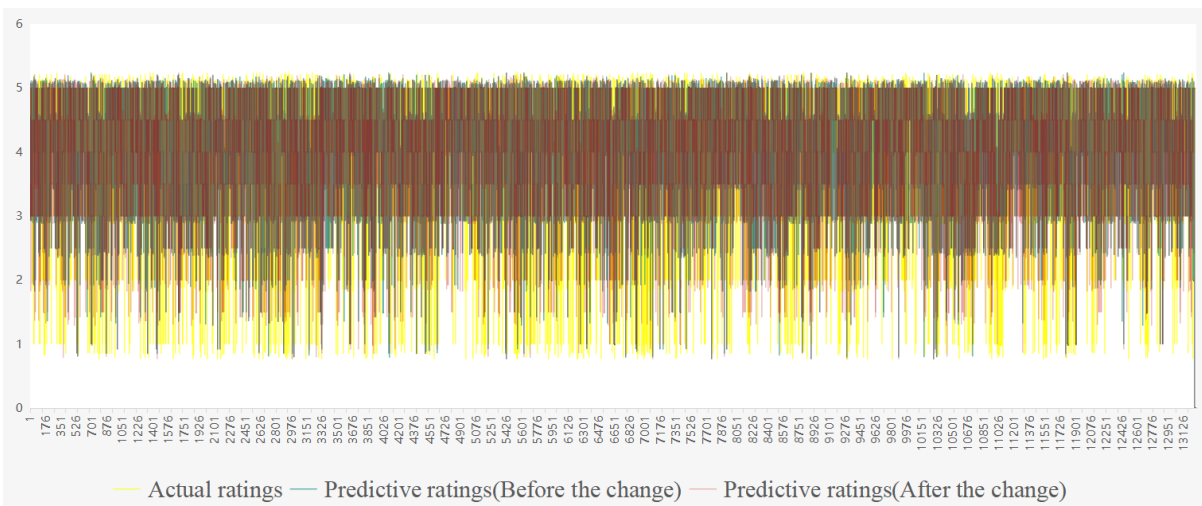


Figure 34: Change vs. No Change Ratings

As shown in Figure 34, qualitative analysis shows that the overall trend of red line and green line is the same and the difference is very small. Monte Carlo quantitative analysis showed that 772 data were randomly changed in the training set of 10260, and the degree of deviation changed from 0.7847929395790902 to 0.8099494606622916, that is, 7.52% of data confusion only caused the degree of deviation to increase by 3.21%. So we can conclude that our model has outstanding model stability.

9 Strengths and Weaknesses

9.1 Strengths

- The LDA-K-TF-IDF model and the naive Bayes model we used are based on solid mathematical principles and are particularly good at dealing with textual data and overcome the limitations of single models;
- When processing the data and selecting the model, it takes into account a variety of situations, compares the models, and corrects the results in time, which optimizes the analysis results to the greatest extent;
- Our model has excellent adaptability and can be generalized to more domains, such as feedback on user comments on social media and analysis of general legal texts.

9.2 Weakness

- When using the LDA-K-TF-IDF model to screen product names, it needs to manually adjust the parameters for many times, which leads to a small number of deviations in the results. In the subsequent improvement, the parameters can be objectively weighted to increase the accuracy.

10 Conclusion

- In question 1, we conducted text analysis on Appendix I and Appendix II, obtained high-frequency words after removing stop words and drew word cloud map;
- In question 2, we carried out semantic analysis on Appendix III and Appendix IV, and predicted the product names based on LDA-K-TF-IDF model. It was concluded that the former mainly included office products, and the latter mainly included Musical Instruments and their accessories;
- In question 3, we conducted sentiment analysis on Appendix 5 and Appendix 6, inferred the commodity rating based on the improved naive Bayes model, and compared with the actual rating, obtained better results;
- In Question 4, we establish the criteria to distinguish machine reviews from customer reviews, design algorithms for verification, and write letters to customers to summarize online shopping suggestions.

References

- [1] Guo J, Ji Y, Woo S, Zhan C. Analyzing E-Commerce Customer Reviews with NLP[J]. Institute for Applied Computational Science, Dec 29, 2020.
- [2] M Blei D, Andrew Y Ng, Michael I. Jordan. Latent Dirichlet Allocation[J]. Journal of Machine Learning Research, 2003, 3:993-1022.
- [3] Zhang Daohai, Yang Chen. Topic Analysis of Logistics Service quality of E-commerce Platform Based on Semantics [J]. Journal of Logistics Science and Technology, 2023, 46(1) : 36-40.
- [4] Sinaga K P, Yang M. Unsupervised K-Means Clustering Algorithm[J]. IEEE Access, 20 April 2020, 8:80716 - 80727.
- [5] Yu Xiaojian, LIU Guo-peng, LIU Jian-lin, XIAO Wei-Lin. Stocks based on LSTM network and text sentiment analysis[J/OL]. Management Science in China, June 9, 2023.
- [6] Dhaduk H. Performing Sentiment Analysis With Naive Bayes Classifier[J]. Data Science Blogathon, July 13, 2021.

Appendices

Here are simulation programmes we used in our model as follow.

Input python source:

```
# For space reasons, we only show the key parts of the code in general here
import pandas as pd
import spacy
import matplotlib
import numpy as np
import gensim
from gensim import corpora
import pyLDAvis.gensim
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import GridSearchCV
from imblearn.over_sampling import RandomOverSampler
# Data processing section
df = pd.read_csv(r"C:\Users\19826\Desktop\Transformed table csv\app3.csv")
df.head()
df = df.dropna(subset=['reviewText'])
df['reviewText'] = df['reviewText'].str.replace("[^a-zA-Z#]", " ")
from nltk.corpus import stopwords
stop_words = stopwords.words('english')
def remove_stopwords(rev):
```



```

    rev_new = " ".join([i for i in rev if i not in stop_words])
    return rev_new
df['reviewText'] = df['reviewText'].apply(lambda x: ' '.join([w for w in x.
    ↪ split() if len(w)>2]))
reviews = [remove_stopwords(r.split()) for r in df['reviewText']]
reviews = [r.lower() for r in reviews]
nlp = spacy.load('en_core_web_sm', disable=['parser', 'ner'])
def lemmatization(texts, tags=['NOUN']):
    output = []
    for sent in texts:
        doc = nlp(" ".join(sent))
        output.append([token.lemma_ for token in doc if token.pos_ in
    ↪ tags])
    return output
tokenized_reviews = pd.Series(reviews).apply(lambda x: x.split())
reviews_2 = lemmatization(tokenized_reviews)
reviews_3 = []
for i in range(len(reviews_2)):
    reviews_3.append(' '.join(reviews_2[i]))
df['reviews'] = reviews_3
print(df['reviews'])
#LDA section
dictionary = corpora.Dictionary(reviews_2)
doc_term_matrix = [dictionary.doc2bow(rev) for rev in reviews_2]
# Creating the object for LDA model using gensim library
LDA = gensim.models.ldamodel.LdaModel
# Build LDA model
lda_model = LDA(corpus=doc_term_matrix,
    id2word=dictionary,
        num_topics=10,
        random_state=100,
        chunksize=1000,
        passes=50)
topics=lda_model.print_topics()
#K-means section
# Create a TfidfVectorizer object
vectorizer = TfidfVectorizer()
# Compute the tf-idf vectors for the text data
tfidf_matrix = vectorizer.fit_transform(df['reviews'])
# Create a KMeans object with the desired number of clusters
kmeans = KMeans(n_clusters=100,n_init=10)
# Fit the KMeans model to the tf-idf vectors
kmeans.fit(tfidf_matrix)
# Naive Bayes part
# Suppose you already have a dataset that contains the review text and ratings
data = pd.read_csv('C:\\Users\\19826\\Desktop\\Transformed table csv\\app4.csv
    ↪ ')
data = data.dropna(subset=['reviewText'])
data = data.dropna(subset=['summary'])
# Extract evaluation text and score
comments = data['reviewText'].tolist()
ratings = data['overall'].tolist()
summary = data['summary'].tolist()
# Converts text data to feature vectors

```

```

vectorizer1 = CountVectorizer()
vectorizer2 = CountVectorizer()
X1 = vectorizer1.fit_transform(comments)
X2 = vectorizer2.fit_transform(summary)
# Oversampling
ros = RandomOverSampler(random_state=0)
X1_resampled, y1_resampled = ros.fit_resample(X1, ratings)
X2_resampled, y2_resampled = ros.fit_resample(X2, ratings)
# Train naive Bayes models
clf1 = MultinomialNB()
clf1.fit(X1_resampled, y1_resampled)
clf2 = MultinomialNB()
clf2.fit(X2_resampled, y2_resampled)
# Read data that needs to be predicted
test_data = pd.read_csv('C:\\Users\\19826\\Desktop\\Transformed table csv\\
    ↪ app6.csv')
test_data = test_data.dropna(subset=['reviewText'])
test_data = test_data.dropna(subset=['summary'])
test_comments = test_data['reviewText'].tolist()
test_summary = test_data['summary'].tolist()
# Convert test data to feature vectors
X_test_c = vectorizer1.transform(test_comments)
X_test_s = vectorizer2.transform(test_summary)
# Predict the score of test data
predicted_ratings1 = clf1.predict(X_test_c)
predicted_ratings2 = clf2.predict(X_test_s)

```

```

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
import pandas as pd
pd.set_option("display.max_colwidth", 200)
import spacy
# libraries for visualization
import matplotlib
import matplotlib.pyplot as plt
matplotlib.use('TkAgg')
df = pd.read_csv(r"C:\Users\19826\Desktop\Transformed table csv\app4.csv")
df.head()
#delete space
df = df.dropna(subset=['reviewText'])
# remove unwanted characters, numbers and symbols
df['reviewText'] = df['reviewText'].str.replace("[^a-zA-Z#]", " ")
from nltk.corpus import stopwords
stop_words = stopwords.words('english')
# function to remove stopwords
def remove_stopwords(rev):
    rev_new = " ".join([i for i in rev if i not in stop_words])
    return rev_new
# remove short words (length < 3)
df['reviewText'] = df['reviewText'].apply(lambda x: ' '.join([w for w in x.
    ↪ split() if len(w)>2]))
# remove stopwords from the text
reviews = [remove_stopwords(r.split()) for r in df['reviewText']]

```

```

# make entire text lowercase
reviews = [r.lower() for r in reviews]
nlp = spacy.load('en_core_web_sm', disable=['parser', 'ner'])
def lemmatization(texts, tags=['NOUN']): # filter noun
    output = []
    for sent in texts:
        doc = nlp(" ".join(sent))
        output.append([token.lemma_ for token in doc if token.pos_ in
            ↪ tags])
    return output
tokenized_reviews = pd.Series(reviews).apply(lambda x: x.split())
reviews_2 = lemmatization(tokenized_reviews)
reviews_3 = []
for i in range(len(reviews_2)):
    reviews_3.append(' '.join(reviews_2[i]))
df['reviews'] = reviews_3
# Create a TfidfVectorizer object
vectorizer = TfidfVectorizer()

# Compute the tf-idf vectors for the text data
tfidf_matrix = vectorizer.fit_transform(df['reviews'])

# Create a KMeans object with the desired number of clusters
kmeans = KMeans(n_clusters=100, n_init=10)

# Fit the KMeans model to the tf-idf vectors
kmeans.fit(tfidf_matrix)

# # Visualize the clustering results using a scatter plot
# plt.scatter(tfidf_matrix[:, 0], tfidf_matrix[:, 1], c=kmeans.labels_)
# plt.show()
# Create a PCA object
pca = PCA(n_components=2)

# Fit the PCA model to the tf-idf vectors
pca.fit(tfidf_matrix.toarray())

# Transform the tf-idf vectors to 2D
tfidf_2d = pca.transform(tfidf_matrix.toarray())

# Visualize the clustering results using a scatter plot
plt.scatter(tfidf_2d[:, 0], tfidf_2d[:, 1], c=kmeans.labels_)
plt.show()

```

```

from nltk import FreqDist
import pandas as pd
pd.set_option("display.max_colwidth", 200)
import numpy as np
import re
import spacy
import gensim
from gensim import corpora
# libraries for visualization
import matplotlib

```

```

matplotlib.use('TkAgg')
import pyLDAvis
import pyLDAvis.gensim
import matplotlib.pyplot as plt
import seaborn as sns
df = pd.read_csv(r"C:\Users\19826\Desktop\Transformed table csv\app4.csv")
df.head()
#delete space
df = df.dropna(subset=['reviewText'])
# remove unwanted characters, numbers and symbols
df['reviewText'] = df['reviewText'].str.replace("[^a-zA-Z#]", " ")
from nltk.corpus import stopwords
stop_words = stopwords.words('english')
# function to remove stopwords
def remove_stopwords(rev):
    rev_new = " ".join([i for i in rev if i not in stop_words])
    return rev_new
# remove short words (length < 3)
df['reviewText'] = df['reviewText'].apply(lambda x: ' '.join([w for w in x.
    ↪ split() if len(w)>2]))
# remove stopwords from the text
reviews = [remove_stopwords(r.split()) for r in df['reviewText']]
# make entire text lowercase
reviews = [r.lower() for r in reviews]
nlp = spacy.load('en_core_web_sm', disable=['parser', 'ner'])
def lemmatization(texts, tags=['NOUN']): # filter noun
    output = []
    for sent in texts:
        doc = nlp(" ".join(sent))
        output.append([token.lemma_ for token in doc if token.pos_ in
            ↪ tags])
    return output
tokenized_reviews = pd.Series(reviews).apply(lambda x: x.split())
reviews_2 = lemmatization(tokenized_reviews)
reviews_3 = []
for i in range(len(reviews_2)):
    reviews_3.append(' '.join(reviews_2[i]))
df['reviews'] = reviews_3
dictionary = corpora.Dictionary(reviews_2)
doc_term_matrix = [dictionary.doc2bow(rev) for rev in reviews_2]
# Creating the object for LDA model using gensim library
LDA = gensim.models.ldamodel.LdaModel
# Build LDA model
lda_model = LDA(corpus=doc_term_matrix,
id2word=dictionary,
                                num_topics=10,
                                random_state=100,
                                chunksize=1000,
                                passes=50)

topics=lda_model.print_topics()
# print(topics)
#This part is the word frequency extraction part
# num_topics = lda_model.num_topics

```

```

# num_words = 3
# for topic in range(num_topics):
#     print(f"Topic {topic}:")
#     topic_words = lda_model.show_topic(topic, topn=num_words)
#     for word, prob in topic_words:
#         print(f"        {word}: {prob}")
# Visualize the topics
vis = pyLDAvis.gensim.prepare(lda_model, doc_term_matrix, dictionary)
pyLDAvis.save_html(vis, 'C:\\Users\\19826\\Desktop\\lda.html')

```

```

from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from imblearn.over_sampling import RandomOverSampler
import pandas as pd

data = pd.read_csv('C:\\Users\\19826\\Desktop\\Transformed table csv\\app4gai3
    ↪ .csv')
data = data.dropna(subset=['reviewText'])
data = data.dropna(subset=['summary'])

comments = data['reviewText'].tolist()
ratings = data['overall'].tolist()
summary = data['summary'].tolist()

vectorizer1 = CountVectorizer()
vectorizer2 = CountVectorizer()
X1 = vectorizer1.fit_transform(comments)
X2 = vectorizer2.fit_transform(summary)

ros = RandomOverSampler(random_state=0)
X1_resampled, y1_resampled = ros.fit_resample(X1, ratings)
X2_resampled, y2_resampled = ros.fit_resample(X2, ratings)

clf1 = MultinomialNB()
clf1.fit(X1_resampled, y1_resampled)
clf2 = MultinomialNB()
clf2.fit(X2_resampled, y2_resampled)

test_data = pd.read_csv('C:\\Users\\19826\\Desktop\\Transformed table csv\\
    ↪ app6.csv')
test_data = test_data.dropna(subset=['reviewText'])
test_data = test_data.dropna(subset=['summary'])
test_comments = test_data['reviewText'].tolist()
test_summary = test_data['summary'].tolist()

X_test_c = vectorizer1.transform(test_comments)
X_test_s = vectorizer2.transform(test_summary)

predicted_ratings1 = clf1.predict(X_test_c)
predicted_ratings2 = clf2.predict(X_test_s)

predicted_ratings = []
for i in range(0, len(predicted_ratings1)):
    predicted_ratings.append(predicted_ratings1[i]*0.5+predicted_ratings2[i]

```

```

    ↪ ]*0.5)
from openpyxl import load_workbook
wb = load_workbook('C:\\Users\\19826\\Desktop\\Transformed table\\app6yu4.xlsx
    ↪ ')
ws = wb.active
for row, entry in enumerate(predicted_ratings, start=2):
    ws.cell(row=row, column=4, value=entry)
wb.save('C:\\Users\\19826\\Desktop\\Transformed table\\app6yu4.xlsx')
sum = 0
c=test_data['overall'].tolist()
for i in range(0, len(predicted_ratings)):
    sum = sum + abs(predicted_ratings[i] - c[i])
inaccuracy = sum / len(predicted_ratings)
print(inaccuracy)

```

```

from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import GridSearchCV
import pandas as pd
import random

train_data = pd.read_csv(r'C:\\Users\\19826\\Desktop\\Transformed table csv\\
    ↪ app4444.csv')
train_data = train_data.dropna(subset=['reviewText'])
train_data = train_data.reset_index(drop=True)
train_data = train_data.dropna(subset=['summary'])
train_data = train_data.reset_index(drop=True)
train_comments = []
train_ratings = []
train_summary = []
for i in range(0, len(train_data)):
    train_comments.append(train_data['reviewText'][i])
    train_ratings.append(train_data['overall'][i])
    train_summary.append(train_data['summary'][i])
test_comments = []
test_ratings = []
test_summary = []
test_data = pd.read_csv(r'C:\\Users\\19826\\Desktop\\Transformed table csv\\app6.
    ↪ csv')
test_data = test_data.dropna(subset=['reviewText'])
test_data = test_data.reset_index(drop=True)
test_data = test_data.dropna(subset=['summary'])
test_data = test_data.reset_index(drop=True)
for i in range(0, len(test_data)):
    test_comments.append(test_data['reviewText'][i])
    test_ratings.append(test_data['overall'][i])
    test_summary.append(test_data['summary'][i])
vectorizer_comments = CountVectorizer()
X_train_comments = vectorizer_comments.fit_transform(train_comments)
X_test_comments = vectorizer_comments.transform(test_comments)

vectorizer_summary = CountVectorizer()
X_train_summary = vectorizer_summary.fit_transform(train_summary)
X_test_summary = vectorizer_summary.transform(test_summary)

```

```

X_train = vectorizer_comments.fit_transform(train_comments)
X1_train = vectorizer_comments.fit_transform(train_summary)
X_test = vectorizer_summary.transform(test_comments)
X1_test = vectorizer_summary.transform(test_summary)
param_grid = {'alpha': [0.5, 1.0, 1.5, 2.0]}

clf = MultinomialNB()
grid_search = GridSearchCV(clf, param_grid, cv=5)

grid_search.fit(X_train_comments, train_ratings)

print(grid_search.best_params_)

best_clf_comments = grid_search.best_estimator_

predicted_ratings_comment = best_clf_comments.predict(X_test_comments)

clf = MultinomialNB()
grid_search = GridSearchCV(clf, param_grid, cv=5)

grid_search.fit(X_train_summary, train_ratings)

print(grid_search.best_params_)

best_clf_summary = grid_search.best_estimator_

predicted_ratings_summary = best_clf_summary.predict(X_test_summary)
inaccuracy_lst = []
min_err = 5
a_min = -1
# for k in range(0, 1000):
#     a = random.random()
# for i in range(0, 101):
predicted_ratings = []
a = 0.5
for m, n in zip(predicted_ratings_summary, predicted_ratings_comment):
    predicted_ratings.append(m*a+n*(1-a))
print(predicted_ratings)

sum = 0
for i in range(0, len(predicted_ratings)):
    sum = sum + abs(predicted_ratings[i]-test_ratings[i])
inaccuracy = sum/len(predicted_ratings)
print(inaccuracy)
# inaccuracy_lst.append(inaccuracy)
from openpyxl import load_workbook
wb = load_workbook('C:\\Users\\19826\\Desktop\\Transformed table\\app666.xlsx'
    ↪ )
ws = wb.active
for row, entry in enumerate(predicted_ratings, start=2):
    ws.cell(row=row, column=1, value=entry)
wb.save('C:\\Users\\19826\\Desktop\\Transformed table\\app666.xlsx')
#     if inaccuracy < min_err:

```



```
#         min_err = inaccuracy
#         a_min = a
# print(min_err)
# print(a_min)

import pandas as pd
from nltk.corpus import stopwords
from collections import Counter
import spacy

df = pd.read_csv(r"C:\Users\19826\Desktop\Transformed table csv\app4.csv",
    ↪ usecols=['asin', 'reviewText', 'summary'])
df['reviewText'] = df['reviewText'].astype(str)
df['summary'] = df['summary'].astype(str)

df['text'] = df.groupby('asin')['reviewText'].transform(lambda x: ' '.join(x))
    ↪ + ' ' + df.groupby('asin')['summary'].transform(lambda x: ' '.join(x))
df = df[['asin', 'text']].drop_duplicates()

df['text'] = df['text'].str.replace("[^a-zA-Z#]", " ")

stop_words = stopwords.words('english')
df['text'] = df['text'].apply(lambda x: ' '.join([w for w in x.split() if w
    ↪ not in stop_words]))

nlp = spacy.load('en_core_web_sm', disable=['parser', 'ner'])
def lemmatization(texts, tags=['NOUN']):
    output = []
    for sent in texts:
        doc = nlp(sent)
        output.append(' '.join([token.lemma_ for token in doc if token.pos_ in
    ↪ tags]))
    return output

df['text'] = lemmatization(df['text'])

def top_words(text, n=10):
    words = text.split()
    word_counts = Counter(words)
    return [word for word, count in word_counts.most_common(n)]

df['top_words'] = df['text'].apply(top_words)

for index, row in df.iterrows():
    print(row['asin'], row['top_words'])
```